

第9章 线性结构

本章的重点是介绍数据结构的概念和线性结构。在计算机上处理的数据一般都存在着一定的内在联系，如何根据这些联系找到合适的存储方法把数据存储起来，并对数据进行有效的处理，是数据结构所要讨论的主要问题。

线性结构是一种最基本的数据结构，主要包括线性表、栈和队列，本章将详细讨论这三种数据结构的概念及其实现。可以把数组看作是线性表的推广，关于数组的情况本章也将进行简单介绍。

9.1 数据结构概述

9.1.1 什么是数据结构

众所周知，计算机是按照人编制的程序自动对数据进行加工的机器。狭义地说，“数据”可以看做是计算机能够识别、存储和处理的符号的集合。数据的基本单位是数据元素，在不同场合下也把它称为元素、结点、顶点、记录等。数据元素的含义非常广泛，在不同的问题处理中具有不同的形式。例如，当统计一本书有多少页时，书中的每一页就是一个数据元素。而统计书中某一页有多少字时，每个字就是一个数据元素。一个数据元素也可以看作是由一个或多个数据项组成，而数据项（亦称为字段、域等）是具有独立含义的数据的最小单位。例如，在一个英文字母表中，可以把每个英文字母作为一个数据元素，在此情况下，数据元素是由一个数据项组成；在一个职工情况表（表9.1）中，可以把每个职工的情况作为一个数据元素，而每个职工情况（包括职工的职工号、姓名、性别、出生年月、民族、职务、职称、工资等）是由多个数据项组成。

表9-1 职工情况表

职工号	姓名	性别	出生年月	民族	职务	职称	工资	...
0001	张红	男	50.10	汉		教授	1020	
0002	李平	女	58.9	汉		副教授	980	
⋮	⋮	⋮	⋮	⋮		⋮	⋮	
n	王力	男	73.7	汉		助教	650	

在实际应用中，总是把具有相同特征的一组数据元素放到一起进行处理。通常，把具有相同性质的数据元素的集合称为数据对象，例如，英文字母表、职工情况表等均为数据对象。在一个数据对象中，各数据元素并不是孤立存在的，而是彼此之间存在着一定的联系，这种联系就是数据结构。对数据结构的讨论是在计算机范畴内进行，必须考虑在计算机上实现对数据的存储和加工时，如何体现出数据元素之间的联系，因此，作为数据结构概念应包括以下三方面的内容。

1. 数据的逻辑结构

数据的逻辑结构只抽象地反映出数据元素之间的逻辑关系，它与数据的存储无关，是独立于计算机的。例如，由表9.1可以看出，表中除第一个数据元素为表头、最后一个数据元素为表尾外，其他每个数据元素的前面都只有一个数据元素，在它们的后面也都只有一个数据元素，各数据元素之间的关系是线性的。在上述讨论中，只考虑了数据元素之间的逻辑关系，而没有考虑它们在计算机中的存储，所以讨论的是数据的逻辑结构。

根据数据元素之间逻辑关系的不同特性，数据的逻辑结构可分成以下四类基本结构。

1) 集合结构：在集合结构中，数据元素之间的关系是“属于同一集合”，集合是数据元素关系极为松散的一种结构。如图9.1(a)所示。图中用圆圈（结点）表示数据元素。

2) 线性结构：在该结构中，数据元素之间存在着一对一的关系，即除第一个数据元素

和最后一个数据元素外，其他每个数据元素都仅有一个直接前驱元素和一个直接后继元素。如图 9.1 (b) 所示。

3) 树形结构：树形结构又称为层次结构，在该结构中，数据元素之间存在着一对多或多对一的关系，即每个元素若有直接前驱元素，只能有一个，但可以有多个直接后继元素。如图 9.1 (c) 所示。

4) 图形结构：图形结构也称为网状结构，在该结构中，数据元素之间存在着多对多的关系。如图 9.1 (d) 所示。

树形结构和图形结构也合称为非线性结构。

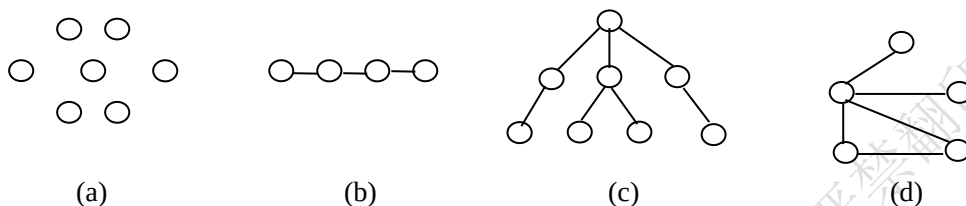


图 9.1 数据的逻辑结构

(a)集合结构；(b)线性结构；(c)树形结构；(d)图形结构

2. 数据的存储结构

数据的存储结构是数据的逻辑结构在计算机存储器中的实现（或称映像），它包括对数据元素的表示和对数据元素之间关系的表示。基本的存储映像方式有以下四种。

(1) 顺序方式

将数据元素按照某种顺序存放一片连续的存储单元内，数据元素之间的逻辑关系是通过它们在存储器中的相对位置来体现的。由于计算机的存储空间是一维的、线性的，所以这种存储方式特别适于存储线性结构。当然，对非线性结构也可以通过某种线性化方法实现顺序存储。

例如，可以把表 9.1 中给出的各职工情况依次存放一片连续的存储空间，此时，元素之间所具有的线性关系是通过它们在物理位置上的相邻来体现的，如图 9.2 所示。

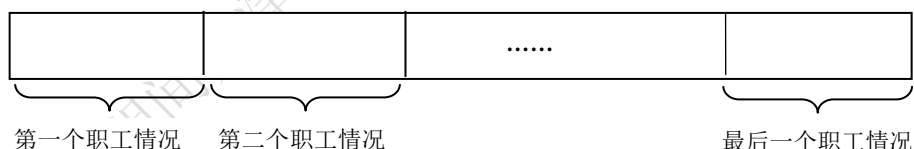


图 9.2 顺序存储方式

顺序存储方式的优点如下：

- ①元素之间的逻辑关系通过它们在存储器中的位置来体现，因此存储密度高。
- ②可以快速地确定出任意元素的存储位置，从而能够实现对元素的随机、快速访问。

顺序存储方式的缺点如下：

- ①插入或删除元素时，可能会引起大量元素的移动，实现效率低。
- ②存储空间需预先分配，如果定的太大会造成空间的浪费，定的太小又可能会发生溢出。

(2) 链接方式

当用任意一组连续的或不连续的存储单元存储数据元素时，数据元素之间的逻辑关系不是靠着它们的存储位置来体现，而是通过附加的指针域表示。在链接方式中，每个数据元素所占用的存储空间通常分为两部分：一部分存放数据元素本身的值（称为数据域）；另一部分存放指示存储地址的指针（称为指针域）。可以设置一个或多个指针域，以指向该数据元素的一个或多个直接后继或直接前驱。对表 9.1 中给出的各职工情况按链接方式进行存储的

情形如图 9.3 所示。

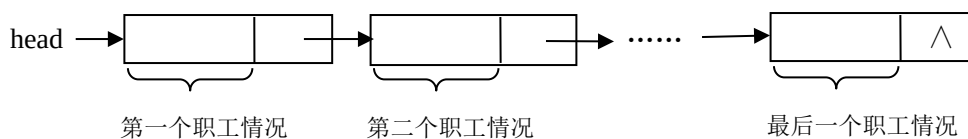


图 9.3 链接存储方式

从图可以看出,自第一个结点开始,有一个指针(用箭头表示)指向它的直接后继结点,直到倒数第一个结点。最后一个结点没有直接后继,所以它的指针域为空,用 $\wedge$ 表示。另外,因为第一个结点没有直接前驱,要设一个头指针 head 指向它。

链接存储方式的优点如下:

①元素之间的逻辑关系通过结点的指针域指出,因此,在插入和删除元素时,不会引起大量元素的移动。

②可动态申请和释放结点空间,所使用的存储空间大小与实际存储的数据元素的数量保持一致。

顺序存储方式的缺点如下:

①不能实现对数据元素的随机访问。

②每个结点中的指针域会耗费一些存储空间。

(3) 索引方式

该方式在存储数据元素的同时建立一个附加的索引表,索引表中的每一行称为一个索引项。它的一般形式为:(关键字,地址)。其中,关键字是指能够唯一标识一个数据元素的一个或多个数据项。利用索引表中索引项可以确定一个数据元素的存储地址。索引方式的优点是检索速度快,缺点是增加的索引表会占用较多的存储空间。另外,在插入和删除数据元素时需要修改索引表,因而会花费更多的时间。

(4) 散列方式

在该方式下,数据元素的存储地址是用它的关键字计算出来的。散列方式又称为哈希表、杂凑表等。优点是检索、增加和删除元素的操作都很快。缺点是,如果使用了不好的散列函数,可能会出现存储单元的冲突,为解决冲突需要额外的时间和空间开销。关于散列函数、“冲突”等概念,详见 11.5.1 的讨论。

以上四种存储方式可以组合使用。例如,在以顺序方式把一组数据存储于大小固定的存储空间时,若存放的数据超出了分配的空间范围,则可用指针指向一溢出区,并把超出的数据存放于溢出区内,这就是顺序与链接相结合的方式。

同一种逻辑结构可以采用不同的存储方式,例如,对线性结构既可以采用顺序方式,又可以采用链接方式、索引方式或散列方式。若同一种逻辑结构采用不同的存储结构,将生成不同的数据结构。

3. 数据的运算

每一种数据结构都有相应的一组运算,用来对结构中的数据进行加工处理。数据运算也称为数据操作。不同的数据结构和应用场合,定义的数据运算也不相同,最常用的运算如下。

- 1) 插入 在数据结构中增添新的数据元素。
- 2) 删除 删除数据结构中某个指定的数据元素。
- 3) 查找 在数据结构中寻找满足特定条件的数据元素及其位置。
- 4) 更新 改变数据结构中某个数据元素的值。
- 5) 排序 按照某种指定的顺序,使数据元素重新排列。

一般来说,在定义数据的逻辑结构时,其对应的各种基本运算也同时被定义好。数据运算的实现方法和效率与采用的存储结构密切相关。因此,在进行数据处理时,选择合适的存

储结构非常重要。

实现任何运算都要给出相应的算法，下面是关于算法的讨论。

9.1.2 算法

算法是对解决问题方法的精确描述，是用来完成某个特定任务的有限步骤序列。有时直接把数据结构中数据运算的实现称为算法。数据运算是在数据逻辑结构之上定义的，而它的具体实现是在计算机上进行的。因此，算法要依赖于数据的存储结构。

1. 算法的基本特征

算法有以下五个基本特征。

- 1) 有穷性 一个算法必须在执行有穷步后结束，并且每一步都能在有限的时间内完成。
- 2) 确定性 算法中的每一条指令必须具有确切的含义，且无二义性。
- 3) 可行性 算法中描述的所有操作都可以通过让已实现的基本运算执行有限次完成。
- 4) 输入 一个算法应该有 0 个或多个输入。这些输入应取自于某个特定的对象集合。
- 5) 输出 一个算法应该有一个或多个输出。这些输出是与输入有某种特定关系的量。

2. 算法的描述

对算法的描述可以有多种方式，例如，自然语言描述、程序设计语言描述、流程图描述等。以下采用 C++ 语言作为描述算法的工具。

3. 算法的评价

在保证算法正确的前提下，评价一个算法主要有以下两个标准。

- 1) 算法的时间复杂度 根据算法编写出的程序在计算机上运行时所消耗的时间。
- 2) 算法的空间复杂度 根据算法编写出的程序在计算机上运行时所需要的存储空间的大小。

一个程序在计算机上运行时所耗费的时间与许多因素有关，而且不可能事先精确算出。一般把程序运行时语句的执行次数作为估算程序执行时间的量度。

例 9.1 程序中语句执行频度的分析。

```

:
:
for(i=0;i<n;i++)      //执行频度为 n+1
    for(j=0;j<n;j++)    //执行频度为 n(n+1)
        A[i][j]=0;      //执行频度为 n^2

```

在例 9.1 的程序段中，各语句的执行频度分别为 $n+1$ 、 $n(n+1)$ 和 n^2 。若用 $f(n)$ 表示所有语句的执行频度，则

$$f(n) = n + 1 + n(n + 1) + n^2 = 2n^2 + 2n + 1$$

当 $n \rightarrow \infty$ 时， $f(n)$ 与 n^2 成正比，即程序运行时间的量度与 n^2 为同一数量级，并记为 $O(n^2)$ 。在数学上，如果 $f(n) = O(g(n))$ ，则称 $f(n)$ 与 $g(n)$ 同阶，或称 $f(n)$ 与 $g(n)$ 为同一数量级，即有

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \neq 0 \quad (c \text{ 是一个常数})$$

把 $f(n) = O(g(n))$ 称做算法的时间复杂度。其中， n 表示问题的规模（大小），一般是一个用来衡量输入数据量大小的整数。例如，在处理一个线性表时，问题的规模可以是该线性表中数据元素的个数。

算法的时间复杂度是一个算法在数据量很大的情况下计算时间的近似量度，它主要用于对功能相近的算法的评价。假设算法 A 和算法 B 是解决同一问题的两个不同算法，算法 A 的时间复杂度是 $O(n)$ ，算法 B 的时间复杂度是 $O(n^2)$ ，尽管它们不能精确给出 A 或 B 需要

的运行时间,但对足够大的 n ,算法A的效率要高于算法B。因为,当 n 趋向无穷大时, n 的增长率比 n^2 慢很多。必须说明的是,对 n 的较小值,算法B的效率可能要好于算法A。按数量级递增排列,常见的时间复杂度有 $O(1)$ 、 $O(\log_2 n)$ 、 $O(n)$ 、 $O(n \log_2 n)$ 、 $O(n^2)$ 、 $O(n^3)$ 、 $O(2^n)$ 等。图9.2显示了各种函数值随 n 的增长而变化的情况。

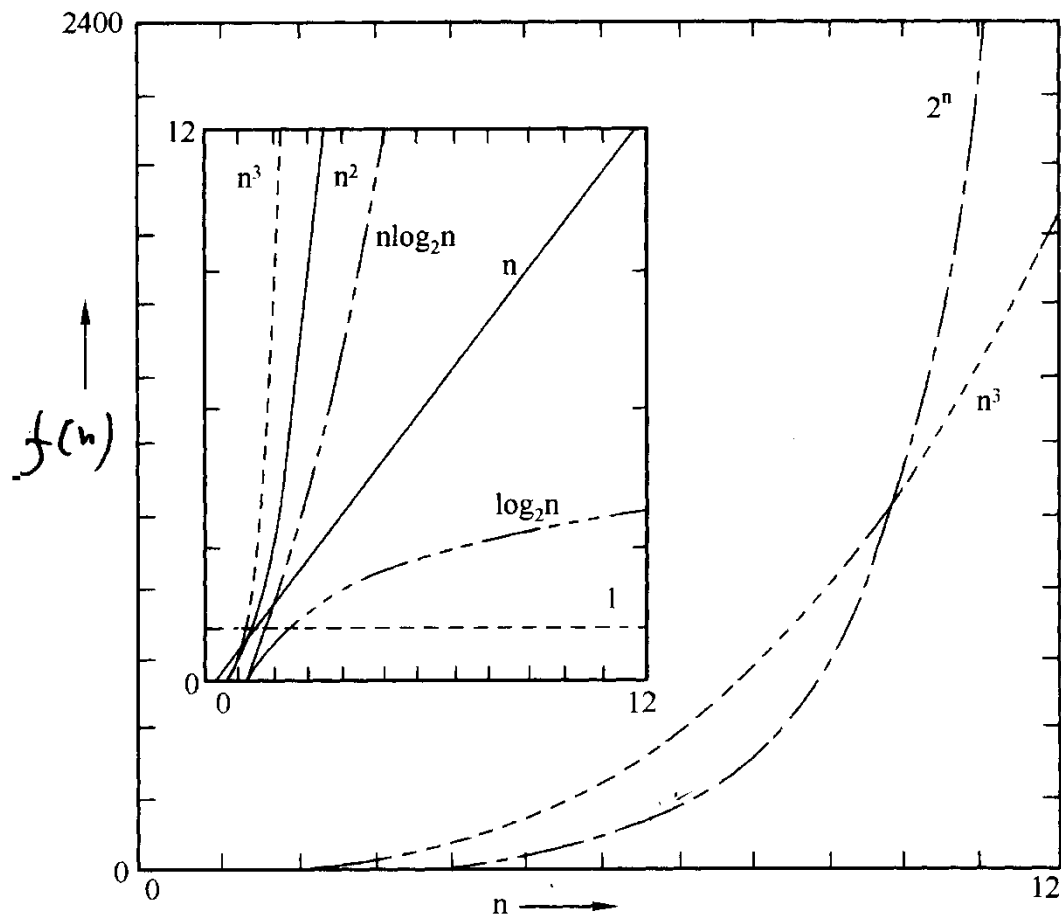


图 9.4 常用时间函数的增长率

有些程序的执行时间不仅依赖于问题的规模,而且与它处理的输入数据集合的状态有关。此时,一般按每种情况发生的概率,求出数学期望值作为算法的平均时间复杂度,或按最坏情况下基本操作的执行频度算出算法在最坏情况下的时间复杂度,以此作为算法的时间复杂度。

一个程序在计算机上运行时所占用的存储空间主要有三部分:一是指令、常量和变量所占用的空间;二是输入数据所占用的空间;三是程序执行时需要使用的辅助空间。程序运行时使用的辅助空间的大小因算法而异,有的算法只需要占用少量的辅助空间,而且不随问题规模的大小而变;有的算法需要的辅助空间的大小随问题规模的增大而增大。因此,通常把程序执行时需要的辅助空间的大小作为分析算法空间复杂度的依据。

与算法时间复杂度的表示一样,算法空间复杂度一般也以数量级形式给出。在通常情况下评价算法只给出时间复杂度就可以了。

9.2 线性表

线性结构是一种最基本、最常用的数据结构。本节讨论的线性表以及后面几节中介绍的栈和队列都属于线性结构。

9.2.1 线性表的逻辑结构

在英文字母表 (A、B、C、……、Z)、职工情况表 (表 9.1) 中, 数据的组织有一个共同的特征, 即它们的组成元素都按某种自然的顺序排成一个序列, 我们把具有这种特征的一组数据元素称为线性表。线性表的抽象描述为: 线性表是由 n ($n \geq 0$) 个相同类型的数据元素 e_0 、 e_1 、……、 e_{n-1} 组成的有限序列。表中数据元素的个数 n 称为表的长度。当 $n = 0$ 时, 线性表为空表。对于一个非空的线性表总可以抽象地写成:

$$(e_0, e_1, \dots, e_{i-1}, e_i, e_{i+1}, \dots, e_{n-1})$$

这里, 数据元素 e_i ($0 \leq i \leq n-1$, 称 i 为数据元素的下标) 只是一个抽象符号。它在不同情况下具有不同含义。

在一个线性表中, 所有数据元素必须具有相同的特征, 即它们都属于同一个数据对象。在线性表的抽象表示中, e_0 为开始元素, 它没有前驱而只有一个直接后继; e_{n-1} 为终端元素, 它没有后继只有一个直接前驱; 其余元素为内部元素, 它们各自都只有一个直接前驱和一个直接后继。

对线性表可以实施的运算较多, 常用的基本运算有:

- ① 创建一个空表;
- ② 判表空;
- ③ 删除表中所有元素;
- ④ 求表长;
- ⑤ 取下标为 i 的数据元素;
- ⑥ 在表中查找与给定值 x 相等的数据元素;
- ⑦ 在下标为 i 的数据元素之前插入一个新的数据元素;
- ⑧ 删除下标为 i 的数据元素。

对线性表还可进行一些更复杂的运算。例如, 按照一定的原则将几个线性表合并成一个线性表, 或将一个线性表拆成几个线性表; 复制一个线性表; 将线性表中的数据元素按照某个数据项值非递增 (或非递减) 顺序排序, 从而得到有序的线性表等等。

9.2.2 线性表的顺序存储结构

在计算机中存储线性表的最简单、最自然的办法是用一组地址连续的存储单元依次地存放线性表中的每一个元素, 通常把用这种方法存储的线性表称为顺序表。

以顺序方式存储线性表时, 逻辑上相邻的元素被存放在相邻的存储单元内, 因此, 可以用元素的物理邻接表示元素的逻辑邻接。也就是说, 在顺序表中数据元素之间的逻辑关系是隐含表示的。

在高级程序设计语言中, 一维数组是采用顺序存储表示的, 所以, 可以用一维数组存储顺序表。把存储线性表的一维数组和实现顺序表各种运算的函数封装在一起, 可以定义顺序表类。例 9.2 给出了一个顺序表类的定义。为简单起见, 这里假定线性表中的所有元素均为字符 (char 类型)。类中定义的私有数据成员 `element` 存放动态分配的一维数组中首元素的地址; `length` 保存当前表的长度, 当表空时, `length` 为 0; `Maxsize` 为顺序表中最多可以存放的元素的个数。

例 9.2 顺序表类

```
#include <iostream.h>
```

```
class SeqList{
```

```
public:
```

```
    SeqList(int m=10);           //构造函数
```

```
    ~SeqList(){delete []element;} //析构函数
```

```
    bool IsEmpty(){return length==0;} //判表是否为空
```

```
    int Length(){return length;} //返回表长
```

```

bool Find(int i,char &x);           //把下标为 i 的元素取至 x
int Search(const char &x);         //返回 x 在表中的下标
bool Insert(int i,const char &x);   //在下标 i 处插入元素 x
bool Delete(int i,char &x);         //返回下标为 i 的元素至 x，并删除之
void ClearList(){length=0;}        //将表清空
void output(ostream &out)const;    //输出表中所有元素的值
friend ostream& operator<<(ostream &out,const SeqList &x);    //重载 "<<"
private:
    int Maxsize;
    int length;
    char *element;
};

```

SeqList 类的构造函数完成一个空表的建立,它根据参数 m 的值动态分配一个一维数组空间,并把该空间的首地址保存在指针变量 element 中。语句如下:

```

SeqList::SeqList(int m){
    element=new char[m];
    Maxsize=m;
    length=0;
}

```

函数 Find()的功能是把下标为 i 的元素取至 x 中。如果元素不存在,函数返回 false,否则返回 true。语句如下:

```

bool SeqList::Find(int i,char &x){
    if(i<0||i>length-1) return false;
    x=element[i];
    return true;
}

```

函数 Search()的功能是在表中查找值与 x 相同的元素,如果找到,返回该元素的下标,否则返回-1。语句如下:

```

int SeqList::Search(const char &x){
    for(int i=0;i<length;i++)
        if(element[i]==x) return i;
    return -1;
}

```

在顺序表中插入或删除元素时,表的长度会发生变化,为保证插入或删除运算完成后,顺序表中的元素的存储顺序和逻辑顺序相一致,必须把表中元素成块移动。

当需要在顺序表指定位置 i 插入一个元素 x 时,必须先把从 i 到 length-1 位置上的所有元素顺序向后移动一个位置,然后在空出的第 i 个位置上插入 x。此外,在实现插入时还会出现两种异常情况,第一种异常是:错误地指定了插入位置 (i<0 或 i>length); 第二种异常是:表的当前长度已达到最大容量,没有空间存放新插入的元素。在以下实现插入的函数中,当出现这两种异常情况时,函数返回 false,否则返回 true。

```

bool SeqList::Insert(int i,const char &x){
    if(i<0||i>length) return false;    //下标越界
    if(length==Maxsize) return false;  //表已满
    for(int k=length-1;k>=i;k--) element[k+1]=element[k];
}

```

```

    element[i]=x;
    length++;
    return true;
}

```

当需要删除顺序表中第 i 个元素时，必须把从 $i+1$ 到 $length-1$ 位置上的所有元素顺序向前移动一个位置。在删除元素时也会出现一种异常情况，即要删除的元素不存在。在以下实现删除的函数中，当出现此异常情况时，函数返回 `false`，否则返回 `true`。

```

bool SeqList::Delete(int i,char &x){
    if(Find(i,x)){
        for(int k=i;k<length-1;k++) element[k]=element[k+1];
        length--;
        return true;
    }
    else return false;    //下标越界
}

```

`Output()`函数用于输出顺序表中所有元素的值。

```

void SeqList::output(ostream &out)const{
    for(int i=0;i<length;i++)
        out<<element[i]<<" ";
}

```

为方便输出，对操作符“<<”进行了重载。

```

ostream& operator<<(ostream &out,const SeqList &x){
    x.output(out);
    return out;
}

```

为方便使用，可将以上 `SeqList` 类的定义及其成员函数的实现存入文件 `seqlist.h` 中。下面给出一个使用 `SeqList` 类的程序。

例 9.3 将一串字符存入一个顺序表，删除其中所有的数字字符。

```

#include "seqlist.h"
void main(){
    char str[]="1C++ 2FORTRAN 3PASCAL 4BASIC";
    SeqList L(100);
    int i;
    for(i=0;str[i]!='\0';i++)
        if(L.Insert(i,str[i])==false){ cout<<"插入异常\n";break;}
    cout<<L<<endl;
    i=0;
    char x ;
    while(i<L.Length()){
        L.Find(i,x);
        if(x>='0'&&x<='9') L.Delete(i,x);
        else i++;
    }
    cout<<L<<endl;
}

```

```
}

```

程序的输出结果为:

```
1C++ 2FORTRAN 3PASCAL 4BASIC
```

```
C++ FORTRAN PASCAL BASIC
```

在顺序表中,元素的存储顺序与逻辑顺序相一致,所以确定表中任一元素的位置非常容易,对表中任一元素的访问也很方便。但如果需要对表进行频繁的插入和删除,则会引起表中元素的大量移动,造成效率的低下。另外,有时很难事先准确地估计出表可能达到的最大长度,如果把它定得太大,会造成空间的浪费,定得太小又可能发生溢出。为了克服顺序表的不足,可采用链接方式存储线性表。

9.2.3 线性表的链式存储结构

通常把以链接方式存储的线性表称为链表。常用的链表有单链表、循环链表和双链表等。

1. 单链表

在单链表中,线性表中的数据元素存放在未必连续的存储单元内。由于元素的存放位置不再反映元素之间的逻辑关系,所以在存储每个元素时,除了要存储元素自身的信息外,还要存储指示其直接后继元素在内存中位置的信息。这两部分信息组成了一个数据元素的存储结构,称它为结点,如图9.5所示。



图 9.5 单链表中的结点

图中, data 域为数据域,用来存放元素自身的值; next 域为指针域,用来存放该元素直接后继结点的地址(或位置)。通过指针域,可以把线性表中的 n 个元素按逻辑顺序链接起来,形成一个链表。因为在此链表中,每个结点只有一个指针,所以称它为单链表。

在单链表中,每个结点的存储地址被存放在其直接前驱结点的指针域中,而开始结点无前驱,所以应设立头指针 head 指向开始结点。同时,由于终端结点无后继,故终端结点的指针域为空(记为 $\wedge$)。当表为空时, head 亦为空。图9.6是一个单链表的示意图。

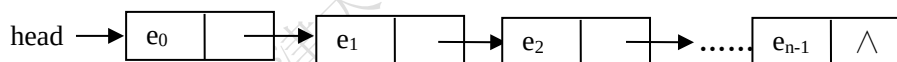


图 9.6 单链表

对图9.6所示的单链表进行插入、删除操作时,必须针对不同的情况采取不同的处理方法,例如,在表头位置进行的插入或删除与在其他位置的处理就不一样,此外,对空表和非空表的处理也有差别。为简化处理,可以在单链表开始结点之前增设一个结点,一般称它为头结点。头结点的数据域可以不存放任何信息,也可以存放线性表的长度等附加信息,而 head 恒指向头结点。引入头结点后,由于开始结点的位置被存放在头结点的指针域中,因此,对它的操作与对表中其他位置上结点的操作没有什么区别,无需做出特殊处理。另外,无论链表是否为空, head 总是一个指向头结点的非空指针,因此,对空表和非空表的处理也是一样的。带头结点的单链表如图9.7所示。在本章以后的讨论中,都采用这种带头结点的单链表。

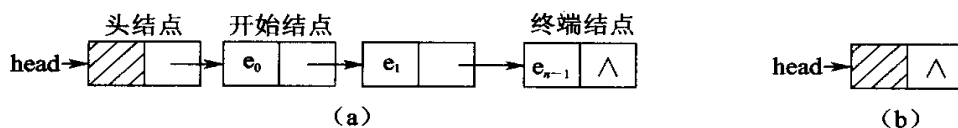


图 9.7 带头结点的单链表
(a) 非空表; (b) 空表

通常，需要两个类来协同表示单链表类，这两个类分别是：单链表的结点类（Node）和单链表类（Chain）。例 9.4 给出这两个类的定义，这里仍假定线性表中的所有元素均为字符（char 类型）。

例 9.4 单链表的结点类（Node）和单链表类（Chain）的定义。

```
#include <iostream.h>

class Chain;           //单链表类 Chain 的前视声明
class Node{           //单链表结点类的定义
    friend class Chain; //定义类 Chain 为友元
private:
    char data;          //结点的数据域
    Node *next;         //结点的指针域
};

class Chain{           //单链表类的定义
public:
    Chain();            //构造函数
    ~Chain();           //析构函数
    bool IsEmpty(){return length==0;} //判表是否为空
    int Length(){return length;}      //返回表长
    bool Find(int i,char &x);          //把下标为 i 的元素取至 x
    int Search(const char &x);         //返回 x 在表中的下标
    bool Insert(int i,const char &x);  //将元素 x 插入表中第 i 个位置
    bool Delete(int i,char &x);        //第 i 个元素送至 x，删除其对应的结点
    void ClearList();                //将表清空
    void output(ostream &out)const;   //输出表中所有元素的值
    friend ostream& operator<<(ostream &out,const Chain &x); //重载 "<<"
private:
    Node *head;                //头指针
    int length;                //表长
};
```

类 Chain 的构造函数用于创建一个空的单链表，具体实现为：

```
Chain::Chain(){
    head=new Node;
    head->next=0;
    length=0;
}
```

在类 Chain 的析构函数中，把链表占有的所有存储空间释放掉，它首先调用 ClearList()成员函数释放除头结点外的所有结点，然后再释放头结点。函数如下：

```
Chain::~~Chain(){
    ClearList();
    delete head;
}
```

函数 Find()的功能是把下标为 i 的元素取至 x 中。如果元素不存在，函数返回 false，否则返回 true。查找表中第 i 个结点时，需要从链头开始沿链接方向逐个查找。函数如下：

```
bool Chain::Find(int i,char &x){
```

```

    if(i<0||i>length-1) return false;
    Node *p=head->next;
    int k=0;
    while(k<i){
        p=p->next;
        k++;
    }
    x=p->data;
    return true;
}

```

函数 Search()的功能是在表中查找值与 x 相同的元素。如果找到, 返回该元素的下标, 否则返回-1。与 Find()函数类似, 查找工作也必须从链头开始沿链接方向逐个进行。函数如下:

```

int Chain::Search(const char &x){
    Node *p=head->next;
    int i=0;
    while(p!=0){
        if(p->data==x) return i;
        p=p->next;
        i++;
    }
    return -1;
}

```

为了在单链表第 i 个结点前插入一个元素值为 x 的新结点, 需要完成以下几个步骤:
 ①找到第 i-1 个结点; ②创建新结点, 并用指针 q 指向它; ③将新结点的数据域值置为 x; ④使新结点的指针域指向第 i 个结点; ⑤使第 i-1 个结点的指针域指向新结点。如图 9.8 所示。
 另外, 对错误地指定了插入位置 ($i<0$ 或 $i>\text{length}$) 的情况, 以下实现插入的函数 Insert()将返回 false, 否则返回 true。语句如下:

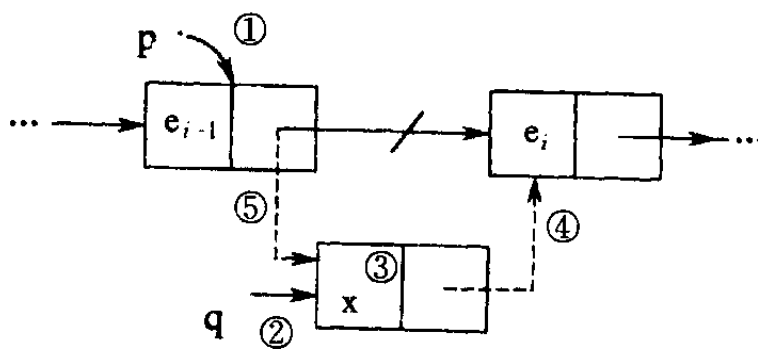


图 9.8 单链表的插入

```

bool Chain::Insert(int i,const char &x){
    if(i<0||i>length) return false;    //下标越界
    Node *p=head;
    int k=0;
    while(k<=i-1){ p=p->next; k++;}
    Node *q;
}

```

```

q=new Node;
q->data=x;
q->next=p->next;
p->next=q;
length++;
return true;
}

```

如果希望删除单链表中第 i 个结点, 应该完成以下步骤: ①找到第 $i-1$ 个结点, 将其地址存入指针变量 p ; ②用指针变量 q 指向第 i 个结点; ③把第 $i+1$ 个结点的地址存入第 $i-1$ 个结点的指针域; ④释放指针变量 q 指向的结点。如图 9.9 所示。当试图删除一个不存在的元素时, 以下实现删除的函数将返回 `false`, 否则返回 `true`。语句如下:

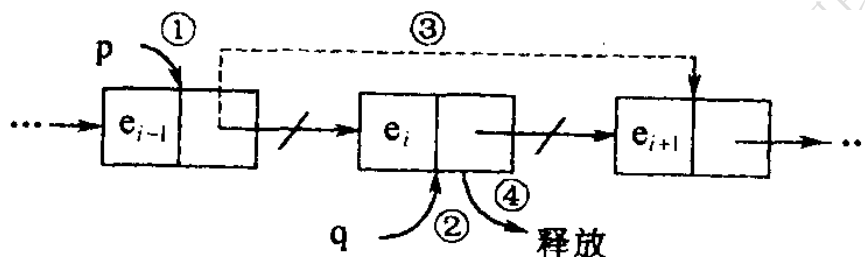


图 9.9 单链表的删除

```

bool Chain::Delete(int i, char &x){
    if(i<0||i>length-1) return false;    //下标越界
    Node *p=head;
    for(int k=0;k<i;k++)p=p->next;
    Node *q=p->next;
    x=q->data;
    p->next=q->next;
    delete q;
    length--;
    return true;
}

```

函数 `ClearList()` 的功能是把表清空, 即将单链表置为初始状态。函数如下:

```

void Chain::ClearList(){
    Node *p=head->next,*q;
    while(p!=0){
        q=p;
        p=p->next;
        delete q;
    }
    head->next=0;
    length=0;
}

```

下面两个函数用于显示表中所有元素的值。

```

void Chain::output(ostream &out)const{
    Node *p=head->next;

```

```

while(p!=0){
    out<<p->data<<" ";
    p=p->next;
}
}
ostream& operator<<(ostream &out,const Chain &x){
    x.output(out);
    return out;
}

```

为方便起见,可将以上两个类 Node 和 Chain 的定义及其成员函数的实现放到文件 chain.h 中。下面给出一个使用单链表类的程序。

例 9.5 将例 9.3 用顺序表实现的程序改用单链表实现。

```
#include "chain.h"
```

```

void main(){
    char str[]="1C++ 2FORTRAN 3PASCAL 4BASIC";
    Chain L;
    int i;
    for(i=0;str[i]!='\0';i++)
        if(L.Insert(i,str[i])!=false) {cout<<"插入异常\n"; break;}
    cout<<L<<endl;
    i=0;
    char x;
    while(i<L.Length()){
        L.Find(i,x);
        if(x>='0'&&x<='9')L.Delete(i,x);
        else i++;
    }
    cout<<L<<endl;
}

```

程序的输出结果为:

```
1C++ 2FORTRAN 3PASCAL 4BASIC
```

```
C++ FORTRAN PASCAL BASIC
```

对照例 9.3 和 9.5 可以发现,除了在设计线性表时使用的类不同以外,两个程序基本相同。这说明类的设计和使用类的应用程序的设计是可以分开进行的。应用程序的设计者可以在完全不了解类规定的实现细节的情况下使用它。

在单链表中,即使知道被访问元素的下标也不能像在顺序表中那样按下标直接访问元素,而只能从链表的头结点出发顺着链接方向逐个结点地往下搜索,相应算法的时间复杂度为 $O(n)$ 。单链表的一个重要特点是实现插入、删除的运算灵活方便。它们都不需要移动结点,只要改变结点指针域的值就可以了。虽然在单链表中插入和删除结点不需要移动结点,但为了找到指定下标的结点仍需执行时间复杂度为 $O(n)$ 的运算,所以,插入和删除算法的时间复杂度仍为 $O(n)$ 。在单链表中,结点空间的分配和释放是动态进行的,所以它对长度不断变化的线性表具有较好的适应性。

2. 单循环链表

单循环链表就是在单链表的基础上将终端结点的指针域改为指向头结点。这样,从单循

环链表的任一结点出发便能访问到链表中的其他结点，如图 9.10 所示。

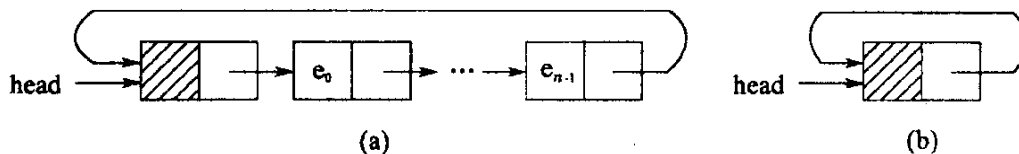


图 9.10 单循环链表

(a) 非空表；(b) 空表

在用头指针表示的单循环链表中，找开始结点 e_0 的时间复杂度是 $O(1)$ ，而要找终端结点 e_{n-1} ，则需从头指针开始遍历整个链表，时间复杂度是 $O(n)$ 。如果改用尾指针 $rear$ 表示单循环链表（图 9.11），则查找开始结点和终端结点都很方便。因此，实际多采取用尾指针表示单循环链表的办法。



图 9.11 设尾指针的单循环链表

3. 双链表

虽然在单链表和单循环链表中可以找到一个已知结点的前驱，但时间耗费较大。若希望从表中快速找到一个结点的前驱，可采用为单链表中的每个结点再增加一个指向其直接前驱指针的办法，这样形成的链表中有两个方向不同的链，故称为双链表，如图 9.12 所示。

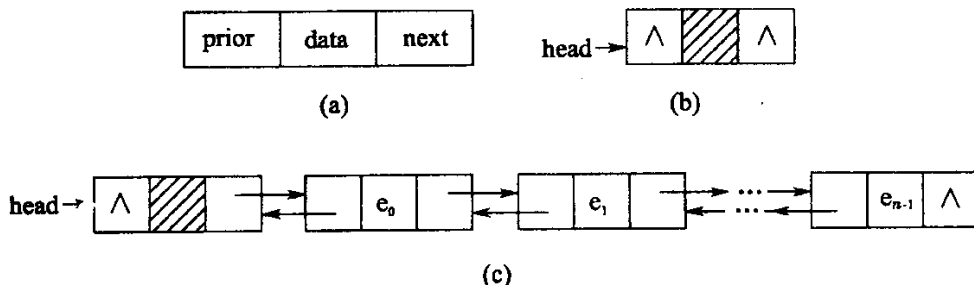


图 9.12 双链表

(a) 结点结构；(b) 带头结点的空双链表；(c) 带头结点的非空双链表

4. 双循环链表

和单循环链表类似，也可以将双向链表的头结点和尾结点链接起来组成双循环链表，如图 9.13 所示。

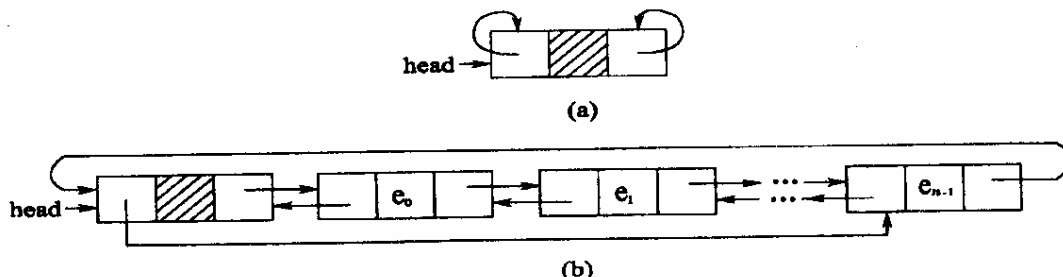


图 9.13 双循环链表

(a) 带头结点的空双循环链表；(b) 带头结点的非空双循环链表

9.3 栈

9.3.1 栈的定义

栈是限定只能在表的一端进行插入和删除运算的线性表。能进行插入和删除的一端称为栈顶，另一端称为栈底。没有任何元素的栈称为空栈。

由栈的定义可知，每次删除的栈元素总是栈中“最新”的元素，即最后进栈的元素，而最先插入的元素被放到栈底，它在最后出栈。在图 9.14 所示的栈中，元素是以 e_0 、 e_1 、……、 e_{n-1} 顺序进栈，而出栈的顺序却是 e_{n-1} 、……、 e_1 、 e_0 。也就是说，栈中元素的变化是按后进先出原则进行的，因此又称栈为后进先出（Last In First Out，简称 LIFO）表。栈在日常生活中经常遇到。例如，有一摞盘子，若规定只能从这摞盘子的顶部取出或放入一个盘子，那它就是一个栈。

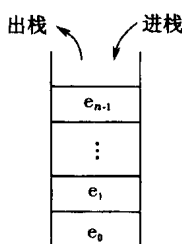


图 9.14 栈

为了体现栈的特点，一般用入栈和出栈表示在栈顶的插入和删除操作。除入栈和出栈外，栈的基本运算还有置栈空、判栈是否为空以及取栈顶元素等。需要说明的是，在使用一个栈时并非要求元素全部进栈后才可以出栈。除了栈空和栈满两个极端条件外，一般情况下，元素的进栈和出栈运算可自由选择。因此，以相同次序入栈的元素序列，可能会由于运算过程不同而导致不同的出栈序列。例如，有 A、B 两个元素依次进栈，可能的出栈序列有两种：① A 进栈后马上出栈，然后 B 进栈后出栈，此时的出栈序列为 A B；② A 进栈后 B 进栈，然后 B 出栈 A 再出栈，此时的出栈序列为 B A。如图 9.15 所示

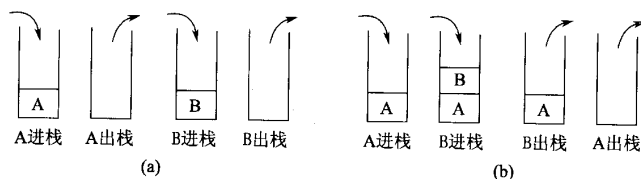


图 9.15 栈元素的不同出栈序列

(a) 出栈序列为 AB； (b) 出栈序列为 BA

由于栈是一种特殊的线性表，因此可以从相应的线性表类中派生出栈类。通过类的派生，可以大大简化程序设计任务，但同时也会使程序的执行效率有明显的降低。栈是一种很基本的数据结构，许多程序都要使用到栈，在下面的讨论中，将直接给出基于顺序存储和基于链接存储的栈类（而不是通过派生得到），这样做可以使执行的效率有较大的提高。

9.3.2 栈的顺序存储结构

栈的顺序存储结构称为顺序栈。顺序栈可以用一维数组实现，因为栈底位置是固定不变的，所以可把栈底设置在一维数组的任意一端。栈顶位置随着进栈和出栈操作不断变化，因此需要一个整型变量 top 指示当前栈顶位置，通常称 top 为栈顶指示器。若把栈底设在一维数组的低端，栈空时， top 为 -1。

在例 9.6 中给出了一个字符型（char 类型）顺序栈类的定义。在此假定栈底位于数组的

低端。类的私有数据成员 `stack` 存放动态分配的一维数组中首元素的地址；`top` 为栈顶指示器，当栈空时，`top` 为-1；`Maxsize` 为顺序栈中最多可以存放的元素的个数。

例 9.6 字符型顺序栈类的定义。

```
#include <iostream.h>
class SeqStack{
public:
    SeqStack(int m=10);           //构造函数
    ~SeqStack(){delete []stack;}  //析构函数
    bool IsEmpty(){return top==-1;} //判栈是否为空
    bool IsFull(){return top==Maxsize-1;} //判栈是否为满
    bool Push(const char &x);      //进栈
    bool Pop();                    //出栈
    bool Top(char &x);             //将栈顶元素取至 x
    void ClearStack(){top=-1;}     //清空栈
private:
    int top;                      //栈顶指示器
    int Maxsize;
    char * stack;
};
```

`SeqStack` 类的构造函数用于建立一个空栈。它根据参数 `m` 的值动态分配一个一维数组空间，并把该空间的首地址保存在指针变量 `stack` 中。同时，令 `top` 为-1，把栈置空。其实现为：

```
SeqStack::SeqStack(int m){
    stack=new char[m];
    Maxsize=m;
    top=-1;
}
```

当完成入栈操作时，如果栈已满，则会产生空间溢出，简称为“上溢”。在以下实现入栈的函数中，如遇“上溢”，函数将返回 `false`，否则返回 `true`。语句如下：

```
bool SeqStack::Push(const char &x){
    if(IsFull()) return false;    //栈满
    top++;
    stack[top]=x;
    return true;
}
```

在完成出栈操作时，如果当前栈为空也会产生空间溢出，简称“下溢”。在以下实现出栈的函数中，如遇“下溢”，函数将返回 `false`，否则返回 `true`。语句如下：

```
bool SeqStack::Pop(){
    if(IsEmpty()) return false;   //栈空
    else {top--; return true; }
}
```

函数 `Top()` 的功能是把栈顶元素取至 `x`。在取栈顶元素时，也会发生“下溢”错，在以下取栈顶元素的函数中，对此采取了与出栈函数相同的处理方法。

```
bool SeqStack::Top(char &x){
    if(IsEmpty()) return false;   //栈空
```

```

    else{ x=stack[top]; return true;}
}

```

为方便使用,可以将以上顺序栈类的定义及其成员函数的实现存放在文件 `seqstack.h` 中。下面是一个使用顺序栈类的应用程序。

例 9.7 栈的应用。将一个非负的十进制整数转换为 B ($2 \leq B \leq 16$) 进制数输出。

可用基数除法把一个十进制整数转换为 B 进制数,即将此十进制数不断除以 B ,逐次求得余数,直至商为 0。将所有余数按后得先写、先得后写顺序列出即可。

可以把每次相除得到的余数以字符形式存入一个栈。当商为 0 时,最后一次相除得到的余数位于栈顶。栈中元素(从栈顶到栈底)组成的串便是所求的结果。在以下程序中,用函数 `Base_conversion()` 实现上述转换,函数的第一个参数是被转换的十进制数,第二个参数是指定的基数,函数中定义的字符数组 `digit` 用于把一个余数转换成指定基数下对应的数字字符。

```

#include "seqstack.h"
void Base_conversion(long x,int base){
    char digit[]="0123456789ABCDEF",remainder,ch;
    SeqStack stk;
    int temp=x;
    while(temp!=0){
        remainder=digit[temp%base];
        if(stk.Push(remainder)==false){cout<<"栈满\n"; return;}
        temp/=base;
    }
    cout<<x<<"对应的"<<base<<"进制数是: ";
    while(!stk.IsEmpty()){
        stk.Top(ch);    cout<<ch;
        stk.Pop();
    }
}
void main(){
    long num;
    int B;
    char response;
    do{
        cout<<"输入一个非负的十进制整数和转换基数 B(2≤B≤16):";
        cin>>num>>B;
        Base_conversion(num,B);
        cout<<endl;
        cout<<"继续吗(Y/N)?";
        cin>>response;
    }while(response=='Y'||response=='y');
}

```

程序运行结果:

输入一个非负的十进制整数和转换基数 B ($2 \leq B \leq 16$): 256 2

256 对应的二进制数是: 100000000

继续吗 (Y/N) ? y

输入一个非负的十进制整数和转换基数 B ($2 \leq B \leq 16$): 256 8

256 对应的 8 进制数是: 400

继续吗 (Y/N) ? y

输入一个非负的十进制整数和转换基数 B ($2 \leq B \leq 16$): 256 16

256 对应的 16 进制数是: 100

继续吗 (Y/N) ? n

在实际应用中, 顺序栈所使用的一维数组的大小较难确定。采用在同一个一维数组中让多个栈共享存储空间的办法, 可以起到相互调节余缺、节省存储空间和降低上溢发生频率的作用。例如, 在有二个栈时, 可把二个栈的栈底分别设置在一维数组的两端, 让二个栈各自向中间延伸 (图 9.16), 这样, 二个栈就可以共享中间的存储空间了。

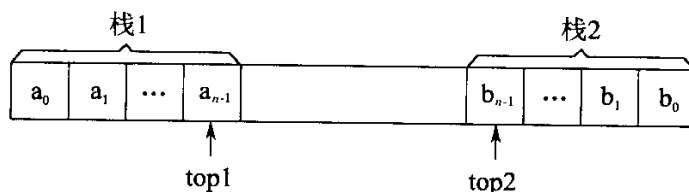


图 9.16 两栈共享存储空间

9.3.3 栈的链式存储结构

栈的链式存储结构称为链式栈。可以用不带头结点单链表表示链式栈, 当然, 要限定所有的插入和删除只能在表头进行。通常, 把链式栈的头指针取名为 top , 并称为栈顶指针。 Top 唯一地确定了一个栈。当 top 为空时, 栈为空。图 9.17 是链式栈的示意图。

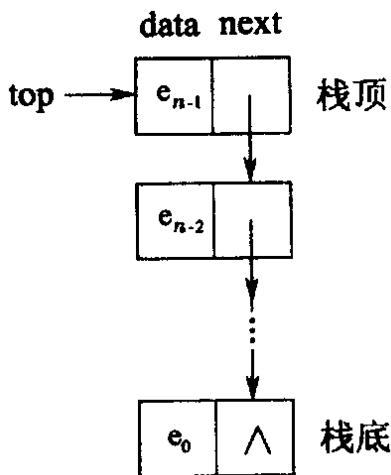


图 9.17 链式栈

链式栈可以看作是单链表的特例。在例 9.8 中只给出了字符型链式栈类及其结点类的定义, 不做过多解释。

例 9.8 字符型链式栈类及其结点类的定义。

```
#include <iostream.h>
class LinkedStack;           //链式栈类 LinkedStack 的前视声明
class Node{                  //链式栈结点类的定义
    friend class LinkedStack; //定义类 LinkedStack 为友元
private:
    char data;                //结点的数据域
    Node *next;               //结点的指针域
```

```

};
class LinkedStack{
public:
    LinkedStack(){top=0;}           //构造函数
    ~LinkedStack();                 //析构函数
    bool IsEmpty(){return top==0;} //判栈是否为空
    void Push(const char & x);      //进栈
    bool Pop();                     //出栈
    bool Top(char &x);              //将栈顶元素取至 x
    void ClearStack();              //清空栈
private:
    Node *top;                     //栈顶指针
};
LinkedStack::~~LinkedStack(){
    ClearStack();
}
void LinkedStack::Push(const char &x){
    Node *p=new Node;
    p->data=x;
    p->next=top;
    top=p;
}
bool LinkedStack::Pop(){
    if(IsEmpty()) return false;    //栈空
    Node *p=top;
    top=top->next;
    delete p;
    return true;
}
bool LinkedStack::Top(char &x){
    if(IsEmpty()) return false;    //栈空
    else{ x=top->data; return true;}
}
void LinkedStack::ClearStack(){
    Node *p;
    while(top){
        p=top->next;
        delete top;
        top=p;
    }
    top=0;
}
}

```

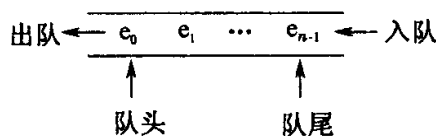
可将上述链式栈类及其结点类定义存入文件 `linkedstack.h` 中。对例 9.7 的数制转换问题也可以用链式栈解决，请自行练习。

栈在计算机中的应用非常广泛,凡是逻辑上需要“后进先出”的操作和过程都用得上栈。如表达式的求值、过程(函数)的调用与返回、递归过程的实现等等,都是栈应用的典型例子。

9.4 队列

9.4.1 队列的定义

队列也是一种运算受限的线性表。它限定所有的插入只能在表的一端进行,所有的删除只能在表的另一端进行。允许删除的一端称为队头(front),允许插入的一端称为队尾(rear),如图 9.18 所示。队列结构在日常生活中并不少见。例如,在排队购物时,排队者必须遵守一条原则,即后到者必须排在队尾,而先来者先购物。



9.18 队列

在队列中,由于新元素总是被加入到队尾,每次删除的总是队头上的元素,即当前“最老的”元素,因此,又称队列为先进先出(First In First Out 简称 FIFO)表。队列中没有元素时的状态称为队空。由于存储空间的限制不能向队中加入元素时的状态称为队满。

队列的插入操作称为入队,删除操作称为出队。除入队和出队外,队列的基本操作还有置队空、判队是否为空及取队头元素等。

队列的存储方法主要有顺序和链接两种。

9.4.2 队列的顺序存储结构

队列的顺序存储结构称为顺序队列。顺序队列实际上是运算受限的顺序表。和顺序表一样,可以用一个一维数组存放当前队列中的元素。由于队列以及队头和队尾的位置总是在变化,因此,需要设置两个整型变量 front 和 rear 分别指示当前队头元素和队尾元素在一维数组中的位置。通常,把 front 称为队头指示器;rear 称为队尾指示器。

为方便起见,这里约定:在 front 中存放实际队头元素的位置,而在 rear 中存放实际队尾元素的下一个位置,即下一个入队元素所在的位置。

设为顺序队列分配的一维数组是 elem, 其长度为 maxsize=4。现在看随着一系列入队和出队操作的进行会发生什么情况。由图 9.19 可以看出,随着入队和出队的进行,队列中的元素逐渐向存储队列的数组的末端移动。当 rear 移出数组时,尽管数组的前端还空着,但也不能执行入队操作,如图 9.19 中的(d)所示。把这种情况称为“假溢出”。解决“假溢出”的方法有两种:一是在发生“假溢出”时将整个队列左移,使队列的首元素位于数组的最前端,并修改队头和队尾指示器。因为需要移动大量的数据,所以,这种方法的效率很低。二是采用循环队列,即把存储队列的一维数组想像成一个首尾相接的圆环,如图 9.20 所示。当循环队列中有元素出队时,队头指示器 front 按顺时针方向进 1;当有元素入队时,队尾指示器 rear 也按顺时针方向进 1。可通过“取余”运算实现 front 和 rear 的循环移动,即

队尾指示器进 1: $rear=(rear+1)\%maxsize$

队头指示器进 1: $front=(front+1)\%maxsize$

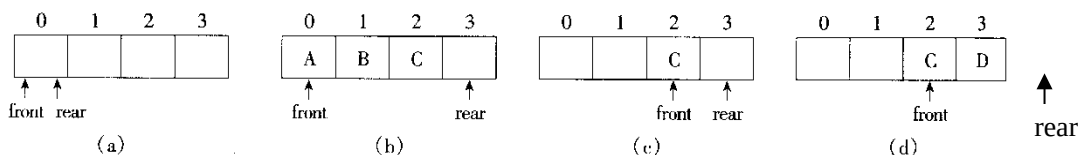


图 9.19 顺序队列

(a) 空队列; (b) A、B、C 入队; (c) A、B 出队; (d) D 入队

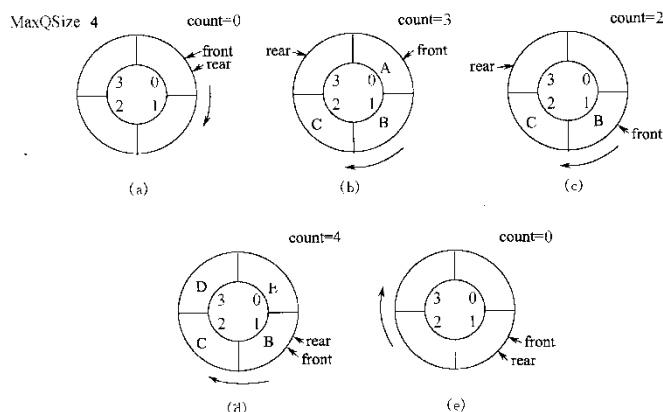


图 9.20 循环队列

(a) 空队列 (b) A、B、C 入队 (c) A 出队 (d) D、E 入队 (e) B、C、D、E 出队

在循环队列情形下,可用整型变量 $count$ 记录当前队列中元素的个数。当 $count$ 为 0 时,队为空;而当 $count$ 等于 $maxsize$ 时,队为满。另外,也可以通过队头指示器 $front$ 和队尾指示器 $rear$ 判断队空和队满。由图 9.20 可以看出,在队空和队满时都有 $rear=front$ 。显然,只凭这个条件还无法判断队列是“空”还是“满”。为区分队空和队满,可以少用一个元素存储空间,把队尾指示器将要追上队头指示器作为队满的条件,即当 $(rear+1)\%maxsize=front$ 时,队为满。而队空的条件仍为 $rear=front$ 。

在例 9.9 给出的字符型循环队列类的定义中,私有数据成员 $elem$ 存放动态分配的一维数组首元素的地址; $maxsize$ 存放该数组中最多可以存放的元素个数; $count$ 记录当前队列中元素的个数; $front$ 和 $rear$ 分别为队头指示器和队尾指示器。

例 9.9 字符型循环队列类的定义。

```
#include <iostream.h>
class Queue{
public:
    Queue(int m=10);           //构造函数
    ~Queue(){delete []elem;}   //析构函数
    bool IsEmpty(){return count==0;} //判队是否为空
    bool IsFull(){return count==maxsize;} //判栈是否为满
    bool EnQueue(const char &x); //入队
    bool DeQueue();           //出队
    bool Front(char &x);      //将队头元素取至 x
    int Length(){return count;} //求队列元素个数
    void ClearQueue();        //清空队列
private:
    int front,rear;
    int count;
    int maxsize;
    char * elem;
};
```

Queue 类的构造函数用来创建一个空队列，其实现如下：

```
Queue::Queue(int m){
    elem=new char[m];
    maxsize=m;
    front=rear=0;
    count=0;
}
```

在以下实现入队的函数中，当队满，不能完成入队操作时，返回 `false`。在成功地向队列中加入元素后，要使队列长度计数器 `count` 增 1，并返回 `true`。语句如下：

```
bool Queue::EnQueue(const char &x){
    if(IsFull()) return false;    //队已满
    elem[rear]=x;
    rear=(rear+1)%maxsize;
    count++;
    return true;
}
```

在以下实现出队的函数中，当队为空，不能完成出队操作时，返回 `false`。在元素出队后，应使队列长度计数器 `count` 减 1，并返回 `true`。语句如下：

```
bool Queue::DeQueue(){
    if(IsEmpty()) return false;    //队已空
    front=(front+1)%maxsize;
    count--;
    return true;
}
```

函数 `Front()` 的功能是，读取队头元素至 `x`。当队空时，不能读取队头元素，对此种情况的处理与出队函数相同。语句如下：

```
bool Queue::Front(char &x){
    if(IsEmpty()) return false;    //队已空
    else {x=elem[front]; return true; }
}
```

以下函数的功能是清空队列：

```
void Queue::ClearQueue(){
    count=0;
    front=rear=0;
}
```

为方便使用，可将 `Queue` 类的定义存于文件 `cirqueue.h` 中。下面给出一个使用循环队列类和顺序栈类的实例。

例 9.10 如果一个字符序列正读倒读均一样，则称它为“回文”。编写判断一个字符序列是否为“回文”的程序。

本程序利用了队列和栈的特性，把字符序列分别装入队列和栈中，然后，比较队头元素和栈顶元素，若不同，则证明不是回文，否则，删除队头元素和栈顶元素，继续上述比较。当队或栈空时，证明是回文。

```
#include "cirqueue.h"
#include "seqstack.h"
```

```

void main(){
    char str[80],x,y;
    Queue que(80);
    SeqStack stk(80);
    cin.getline(str,80);
    for(int i=0;str[i];i++){
        if(que.Enqueue(str[i])==false){cout<<" 队已满\n" ;return; }
        if(stk.Push(str[i])==false){cout<<" 栈已满\n "; return;}
    }
    while(!que.IsEmpty()){
        que.Front(x); stk.Top(y);
        if(x!=y){
            cout<<"不是回文! \n";
            return;
        }
        que.DeQueue(); stk.Pop();
    }
    cout<<"是回文! \n";
}

```

9.4.3 队列的链式存储结构

队列的链式存储称为链式队列，可用只允许在表头删除和在表尾插入的单链表实现，不过需要设置两个指针，指向队头结点的指针 `front` 和指向队尾结点的指针 `rear`。（如图 9.21）所示。初始时，`front` 和 `rear` 均为空。

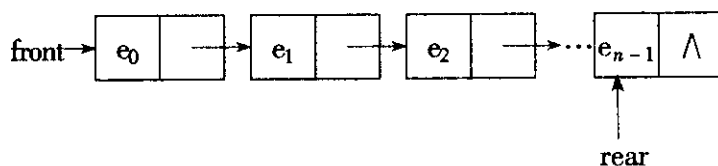


图 9.21 链式队列

下例给出了一个字符型链式队列类及其结点类的定义。为方便起见，使用了整型变量 `count` 记录当前队列中所含的元素个数。

例 9.11 字符型链式队列类及其结点类的定义。

```

#include <iostream.h>
class LinkedQueue;           //链式栈类 LinkedQueue 的前视声明
class Node{                  //链式队结点类的定义
    friend class LinkedQueue; //定义类 LinkedQueue 为友元
private:
    char data;
    Node *next;
};
class LinkedQueue{
public:
    LinkedQueue();           //构造函数
    ~LinkedQueue();         //析构函数

```

```

    bool IsEmpty(){return count==0;}           //判队是否为空
    void EnQueue(const char &x);               //入队
    bool DeQueue();                           //出队
    bool Front(char &x);                      //将队头元素取至 x
    int Length(){return count;}               //求队列元素个数
    void ClearQueue();                         //清空队列
private:
    Node *front,*rear;
    int count;
};

```

LinkedList 类的构造函数用于创建一个空队列，它除了要把队头指针和队尾指针设置为空外，还要把队列长度计数器归 0。其实现为：

```

LinkedList::LinkedList(){
    front=rear=0;
    count=0;
}

```

在 LinkedList 类的析构函数中应释放队列占有的空间，它的功能与清空队列相似，所以直接调用了清空队列函数 ClearQueue()：

```

LinkedList::~~LinkedList(){
    ClearQueue();
}

```

在完成入队操作时，若当前队列为空，新入队的结点既是队尾结点也是队头结点，此时应调整 front 指针。若当前队列非空，新入队的结点被追加到队尾。无论怎样，rear 都应被调整为指向新入队的结点。实现入队的成员函数如下：

```

void LinkedList::EnQueue(const char &x){
    Node *p=new Node;
    p->data=x;
    p->next=0;
    if(!IsEmpty())rear->next=p;
    else front=p;
    rear=p;
    count++;
}

```

队空时，不能进行出队和读队头元素操作。在以下实现出队和读队头元素的函数中，对此情况的处理方法是返回 false。

```

bool LinkedList::DeQueue(){
    if(IsEmpty()) return false;           //队已空
    Node *p=front;
    front=front->next;
    delete p;
    count--;
    return true;
}

bool LinkedList::Front(char &x){

```

```

        if(IsEmpty()) return false;        //队已空
        else {x=front->data; return true; }
    }
ClearQueue()函数的功能是使链式队列恢复为其初始状态。
void LinkedQueue::ClearQueue(){
    Node *p;
    while(front){
        p=front->next;
        delete front;
        front=p;
    }
    front=rear=0;
    count=0;
}

```

9.5 数组（略）

9.6 例题

例 9.12 设 A、B 和 C 均为 seqList 的对象，假定 A 和 B 中的元素都是按递增顺序排列。编写一个函数 Merge()，把 A 和 B 中所有元素放入 C 且使 C 中元素仍按递增顺序排列。

```

#include "seqlist.h"
void Merge(SeqList & A, SeqList & B, SeqList & C){
    int al = A.Length();
    int bl = B.Length();
    int length = al + bl;        //结果表的长度
    int i=0,j=0,k=0;
    char x,y;
    C.ClearList();                //将结果表清空
    while((i<al)&&(j<bl)){
        A.Find(i,x);
        B.Find(j,y);
        if (x<=y){ C.Insert(k++,x); i++; }
        else{ C.Insert(k++,y); j++; }
    }
    // 将 A 或 B 中剩余元素存入 C
    if(i==al)                    // 已处理完 A 中所有元素
        for(int q=j;q<bl;q++,k++){
            B.Find(q,y);
            C.Insert(k,y);
        }
    else
        for(int q=i;q<al;q++,k++){
            A.Find(q,x);
            C.Insert(k,x);
        }
}

```

```

    }
}
void main(){
    SeqList A(20), B(20), C(40);
    for(int i=0;i<10;i++) A.Insert(i, 'b'+2*i);
    for(i=0;i<5;i++) B.Insert(i,'a'+2*i);
    cout << "第一个表中的元素是: ";
    cout << A << endl;
    cout << "第二个表中的元素是: ";
    cout << B << endl;
    Merge(A,B,C);
    cout << "结果表中的元素是: ";
    cout << C << endl;
}

```

程序运行结果:

第一个表中的元素是: b d f h j l n p r t

第二个表中的元素是: a c e g i

结果表中的元素是: a b c d e f g h i j l n p r t

例 9.13 在单链表类 Chain 中增加一个成员函数 hInsert(), 其功能是在表头插入一个新元素。

```

void Chain::hInsert(const char &x){
    Node *p=new Node;
    p->data=x;
    p->next=head->next;
    head->next=p;
    length++;
}

```

例 9.14 编写程序匹配一个字符串的左、右括号。例如, 字符串“(c*(a+b) -d)”在位置 1 和 4 的左括号分别与位置 8 和 11 的右括号相匹配。对于字符串“(x-y))”(“”, 位置 6 和 7 的括号都没有与之相匹配的括号。

栈是解决这个问题的最有效的数据结构。当从左向右扫描一个字符串时, 每个右括号将与最近遇到的那个未匹配的左括号相配对, 据此, 可以在扫描过程中把所遇到的左括号压入栈, 每当扫描到一个右括号时, 就将它与栈顶的左括号(如果存在的话)相匹配, 同时从栈顶删除该左括号。在以下实现匹配的程序中使用了在 9.3.3 节定义的链式栈。

```

#include "linkedstack.h"
#include <string.h>
void printMatchedPairs(char expr[]){
    LinkedStack s;
    int length =strlen(expr);
    char x;
    for (int i = 0; i < length; i++)
        if (expr[i] == '(') s.Push(i);
        else
            if (expr[i] == ')')
                if(s.Top(x)==true){

```

```
        cout<<(int)x<<' ' << i << endl;
        s.Pop();
    }
    else{    // 栈空，不匹配
        cout << "No match for right parenthesis"
        << " at " << i << endl;
    }
    // 不能匹配栈中存留的 '('
    while (!s.IsEmpty()){
        s.Top(x);
        cout << "No match for left parenthesis at " << (int)x << endl;
        s.Pop();
    }
}

void main(){
    char expr[80];
    cout << "Type an expression" << endl;
    cin >> expr;
    cout << "The pairs of matching parentheses in"
        << endl << expr << endl << "are" << endl;
    printMatchedPairs(expr);
}
```

练习 9

一、单项选择题

- _____ 是数据的基本单位。
A) 数据元素 B) 数据项 C) 数据对象 D) 一个字母
- 数据对象是具有相同性质的 _____ 的集合。
A) 数据项 B) 数据元素 C) 数据 D) 数据结构
- _____ 又合称为非线性结构。
A) 集合结构和图形结构 B) 集合结构和树形结构
C) 集合结构和线性结构 D) 树形结构和图形结构
- 算法的有穷性是指 _____。
A) 输出的数据是有限的 B) 描述算法的语句是有限的
C) 所需要的执行时间是有限的 D) 所需要的存储空间是有限的
- 以下关于线性表错误的说法是 _____。
A) 线性表可以为空 B) 线性表的长度可变 C) 线性表中元素个数有限
D) 线性表中元素个数固定
- 顺序表是线性表的 _____。
A) 逻辑结构 B) 顺序存储结构 C) 链接存储结构 D) 与线性表无关
- 设 p 指向单链表中的一个结点, 如果要把由 q 指向的结点插入到 p 所指向的结点的后面, 需要的操作是 _____。
A) $p \rightarrow next = q; q \rightarrow next = p \rightarrow next;$ B) $q \rightarrow next = p \rightarrow next; p = q;$
C) $q \rightarrow next = p \rightarrow next; p \rightarrow next = q \rightarrow next;$ D) $q \rightarrow next = p \rightarrow next; p \rightarrow next = q;$
- 与单链表相比, 双链表具有的优点是 _____。
A) 可实现随机存取 B) 占用更少的存储空间
C) 访问某结点的前驱速度更快 D) 插入、删除操作更方便
- 对于一个栈, 设输入序列为 1, 2, 3, 4, 5, 则 _____ 不可能是栈的输出序列。
A) 2, 3, 4, 1, 5 B) 5, 4, 1, 3, 2 C) 2, 3, 1, 4, 5
D) 1, 5, 4, 3, 2
- 程序设计中“函数的调用与返回”一般通过下述哪种数据结构形式实现 _____。
A) 链表 B) 栈 C) 队列 D) 顺序表
- 在完成出队或读队头元素操作时, _____。
A) 必须判断队是否为空 B) 必须判断队是否为满
C) 要判别队列元素的类型 D) 不必做任何判断
- “假溢出”现象会在 _____ 中发生。
A) 顺序栈 B) 链式队列 C) 顺序队列 D) 顺序表
- 多维数组各维的 _____。
A) 上界固定, 下界不固定 B) 上、下界都是固定的
C) 上界不固定, 下界固定 D) 上、下界都不固定
- 将二维数组 A[10][20] 中元素按列优先顺序存储到一维数组 B 时, 数组元素 A[4][9] 在 B 中的对应元素是 _____。
A) B[36] B) B[13] C) B[89] D) B[94]

二、简答题

- 简述下列术语: 数据、数据元素、数据对象、数据结构、逻辑结构、存储结构、线性结构、非线性结构。

2. 举一个数据结构的例子，叙述其逻辑结构、存储结构和运算这三方面的内容。
3. 假设一个包含字母 A、B、C、E、F、G 的线性表存储在一块地址相连的存储单元中，按字母顺序插入字母 D 时需要的动作是什么？
4. 对线性表可用顺序方式和链接方式进行存储，请比较两种存储方式各自的优缺点，并说明在何种情况下采用何种存储方式为宜。
5. 带头结点的单链表和不带头结点的单链表在实现数据元素的插入和删除运算时有何不同？
6. 设有编号为 1、2、3 的三辆列车，顺序开进一个栈式结构的站台，请写出这三辆车开出车站时的所有可能顺序。
7. 在顺序队列中，什么是“真溢出”？什么是“假溢出”？为什么顺序队列通常都采用顺序循环队列结构？
8. 在顺序循环队列中，设以队尾指示器 rear 将要追上队头指示器 front 作为队满的条件，请写出通过 front 和 rear 求队列中元素个数的式子。
9. 假设栈 S 中有 N 个元素。若只借助一个队列，能否将 S 中 N 个元素的顺序颠倒？若可行，请简略描述操作步骤。
10. 将矩阵 A 按“列优先”顺序存储，设每个元素占 C 个存储单元，已知 $a_{0,0}$ 的存储地址为 D，写出元素 $a_{i,j}$ ($0 \leq i \leq m-1$, $0 \leq j \leq n-1$) 的地址计算公式。
11. 一个 $n \times n$ 的对称矩阵，如果按“行优先”或“列优先”顺序进行压缩存储，则需要的存储空间容量是多少？
12. 写出稀疏矩阵 X (图 9.25) 的三元组表。

$$X = \begin{bmatrix} 15 & 0 & 0 & 22 & 0 & -12 \\ 0 & 11 & 5 & 0 & 0 & 0 \\ 0 & 0 & 0 & -7 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 87 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 \end{bmatrix}$$

图 9.25 稀疏矩阵

三、程序填空

1. 以下函数实现的功能是，使用 SeqList 的成员函数 Find 和 Chain 的成员函数 Insert，把顺序表 A 转换成单链表 B。

```
#include "seqlist.h"
#include "chain.h"
void SeqListtoChain(SeqList &A, Chain &B){
    char x;
    _____ ;    //将表 B 清空
    int i=0;
    while(i< _____ ){
        A.Find(i,x);
        B.Insert(i,x);
        _____ ;
    }
}
```

2. Reverse 是在单链表类 Chain 中增加的成员函数，它的功能是在原地实现表元素的反转。

```
void Chain::Reverse(){
```

```

Node *current = head->next,      //指向当前处理的结点
      *next,                      // 指向当前结点的直接后继
      *last = 0;                  // 指向反转后当前结点的直接后继
while ( _____ ) {
    next = current->next;
    current->next = last;
    last = current;
    current = _____ ;
}
_____ = last;                    //last 指向反转后链表的开始结点
}

```

3. 反转函数 Reverse 有两个参数 A 和 B, A 作为输入的链表, B 是把 A 反转后得到的链表, 完成反转后, A 变为空表。

```

#include "chain.h"
void Reverse(Chain &A, Chain &B){
    B.ClearList();
    char x;
    int i, len;
    len = A.Length();
    for(i=0; i<len; i++){
        A.Find( _____ , x);
        B.Insert(i, x);
    }
    _____ ;
}

```

四、算法设计题

1. 在顺序表类 SeqList 中增加一个实现反转的成员函数, 即将原来的数据元素序列 ($e_0, e_1, \dots, e_{n-2}, e_{n-1}$) 变为 ($e_{n-1}, e_{n-2}, \dots, e_1, e_0$), 要求反转操作就地进行 (即在数组 element 本身的空间内进行)。
2. 在顺序表类 SeqList 和单链表类 Chain 中分别增加一个成员函数, 其功能是删除表中所有等于 item 的数据元素。
3. 在 SeqList 类中增加一个成员函数 Half(), 它能删除顺序表中所有下标为偶数的元素。
4. 编写程序求出顺序表中最大元素及其在表中的位置。
5. 借助于栈完成程序填空中的第 2 题。
6. 在单链表类 Chain 中增加一个成员函数 rInsert(), 其功能是把一个新元素插到表尾。
7. 重写 SeqList 和 Chain 的插入成员函数, 函数原型为: `void Insert(const T &x);` 使用该函数完成插入后, 线性表中元素按递增顺序排列。
8. 设两个栈共享存储空间 (图 9.16), 试编写相关的栈类。
9. 编写一个程序, 读入一个英文字母串, 将其中的大写字母插入第一个队, 小写字母插入第二个队, 然后分别输出这两个队中的元素。